

LG

Contents:Purpose:Purpose:

- Build a traffic signal controller that uses timed phases and car detection inputs to decide when traffic should continue in the current direction and when the intersection should change directions. Normal operation uses a 5 second green interval and a 1 second yellow transition. Later in the lab, car detection sensors allow the controller to respond to traffic activity instead of changing directions blindly at the end of every green interval. Using hardware timers to create the required timing and explore the idea of a state machine to keep track of what the intersection is doing and what has happened during the current timing interval.

- Part 4 begins with a smaller timer exercise so you can measure timer-interrupt behavior directly, before applying timers to the intersection.

Standards and references:

- RCET 3375 Lab Standard
- RCET PIC-AS Style Guide
- RCET 3375 PIC16F883 Timers
- RCET Flowchart Guide
- PIC16F883/882/884/886/887 Data sheet
- PIC micro Midrange MCU Family Reference Manual
- previous RCET 3375 lab book documentation.

Equipment and materials:

- MPLAB X IDE and Pic-as toolchain
- PICkit programmer/debugger
- PIC16F883 circuit
- 4MHz crystal oscillator circuit
- 14 LED's various colors
- current limiting resistors
- 3 momentary pushbutton switches as needed
- one SPST switch (Single Pull single throw)
- oscilloscope
- frequency counter or logic analyzer optional
- breadboard, jumpers, and interface components as required
- Lab book.

LG

Part 1 - 20 ms Timer Proof of life:

Goal:

- Configure one PIC16F883 hardware timer to generate a periodic interrupt every 20 ms nominal. Use two diagnostic PORTA outputs to measure both the timer interrupt and the effect of the interrupt on the main loop execution

- For the course 4MHz oscillator:

$$F_{OSC} = 4\text{MHz}$$

$$F_{CY} = F_{OSC}/4 = 1\text{MHz}$$

$$T_{CY} = 1\mu\text{s}$$

Required diagnostic signals:

- Choose two available digital PORTA outputs and document them.

- Timer diagnostic pulse

For every interrupt:

- 1) service the timer as required by design.
- 2) drive the timer diagnostic output high
- 3) drive it low on the immediately following instruction.

- The pulse should be as short as practical. The two output instructions should be adjacent, equivalent to

• BCF PORTA, X

• BSF PORTA, X

- This produces one short diagnostic pulse every 20 ms.

- Main-loop diagnostic

• In main, continuously toggle the second PORTA diagnostic bit with no intentional delay. The main-loop signal should make it easy to see when and where normal execution is interrupted and where it resumes.

Timer-period boundary:

- Define exactly what marks the beginning of a new timer period for your chosen timer. Place the rising edge of the timer diagnostic pulse as close as practical to that period boundary. Then determine exactly how far the diagnostic edge leads or lags the timer-period boundary. Do not assume the GPIO edge and the hardware timer event occur at the same instant.

Required behavior:

- 20ms periodic timer interrupt
- One very short PORTA diagnostic pulse per timer interrupt
- A Second PORTA diagnostic bit toggling continuously in main
- Correct interrupt context save/restore
- Correct timer flag service and reload/configuration behavior.
- No software-delay loop used to generate the timer period.
- Main resumes normally after every timer interrupt.

Lab 6 - Timers

• Before Lab:

- timer choice and reason
- complete timer calculation
- timer and interrupt SFR documentation
- timer-periodic boundary definition
- predicted timer diagnostic period
- predicted instruction-cycle offset from the timer-period boundary to the diagnostic rising edge
- selected PORTA diagnostic pins
- main/ISR flowchart
- source code

• In the Lab:

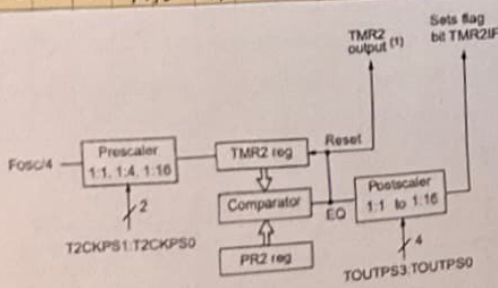
- 1) Verify the main-loop diagnostic signal before enabling the timer interrupt
 - 2) Enable the timer and verify one timer diagnostic pulse every 20 ms.
 - 3) Display both PORTA diagnostic signals on the oscilloscope
 - 4) Observe where timer interrupt service disturbs the otherwise continuous main-loop diagnostic activity.
 - 5) Measure the timer diagnostic pulse spacing.
 - 6) Compare measured timing with the predicted 20ms period.
 - 7) Compare the observed diagnostic rising edge with calculated timer-period boundary.
 - 8) Explain the instruction path that creates the measured lead or lag.
- The shortest diagnostic pulse may be difficult to trigger on or display clearly. If measurement limitations required to temporarily widen the pulse, change the test conditions, or use another instrument configuration, document:

- what was difficult to measure
- what you changed
- why the change was justified
- whether the change affects the timing quantity being evaluated.

$$\text{Timer 2} = \text{period} \rightarrow \left(\text{prescaler} \times (PR2 + 1) \times \text{postscaler} \right)$$

$$20,000 / 16 = 1.25K / 5 = 250$$

- Values to get
- prescaler = 1:16
 - postscaler = 1:5
 - (PR2 + 1) = 250
 - PR2 = 249



Note: TMR2 register output can be software selected by the SSP Module as a baud clock.

• Figure 1: Timer 2 Block Diagram -
Doc # DS40001291H - page 83

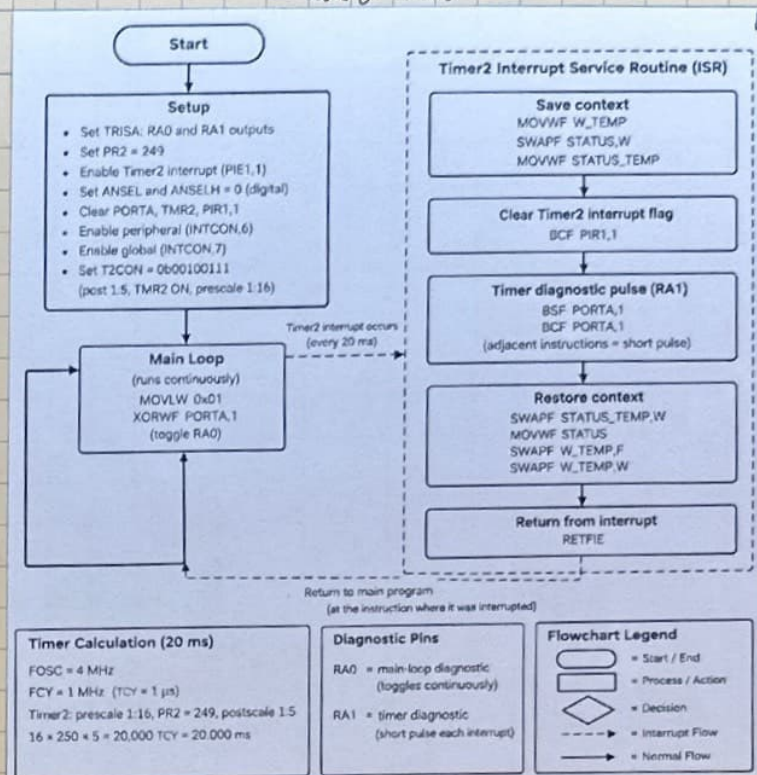


Figure 2: Flowchart Part 4 20ms Timer Proof of Life

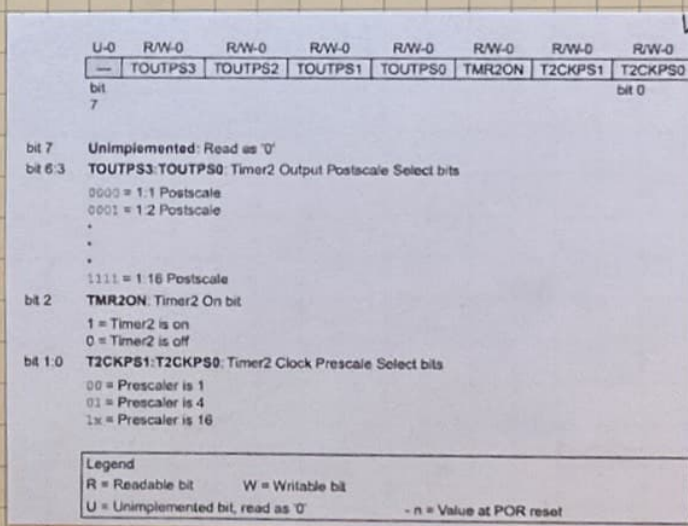


Figure 3: T2CON - Timer2 control register

This was the control register to set up timer 2 to influence the interrupt which is enabled in PIE1 bit 1. This enables the Timer2 to PR2 match interrupt. The flag that will be reset is in the PIR1 bit 1 has the interrupt flag bit. These were the newer registers used in this first part see pg 86 for SFR documentation and ISR code.

Setup:

```

; Bank 1
BANKSEL TRISA
BCF TRISA,0 ;Set RAO as output for main loop
BCF TRISA,1 ;Set RAL as output for timer diagnostic
MOVLW 249 ;PR2 = 249 counts on Timer match } move 249 to make the 20ms count period
MOVWF PR2
BSF PIR1,1 ;enable Timer2 to PR2 match interrupt enable

```

```

; Bank 3
BANKSEL ANSEL
CLRF ANSEL
CLRF ANSELN

```

```

; Bank 0
BANKSEL PORTA
CLRF PORTA ;monitor pin starts low
CLRF TMR2 ;Timer2 count starts at 0
BCF PIR1,1 ;Clear Timer2 interrupt flag →

```

```

BSF INTCN,6 ;enable peripheral } set interrupts
BSF INTCN,7 ;global enable

```

```

MOVLW 0b00100111 ;Turn on postscaler 1:5, Timer2 On, prescale 1:16 → This is how to set binary count.
MOVWF T2CON

```

Main:

```

MOVLW 0x01 ;loads RAO to toggle } toggle RAO in main to see interrupt.
XORWF PORTA,1 ;toggle RAO
GOTO Main

```

Figure 4: SFR Setup and Main code

ISRHandler:

```

; save
MOVWF W_TEMP ;save W (no flags affected)
SWAPF STATUS,W ;STATUS into W with nibbles swapped, no flags change } Save
MOVWF STATUS_TEMP ;save swapped STATUS

; only reset Timer2
BANKSEL PIR1
BCF PIR1,1 ;clear Timer2 to PR2 interrupt flag
BSF PORTA,1 ;save timer diagnostic pulse high } clear flag and show that something happened
BCF PORTA,1 ;set timer diagnostic pulse low

```

ISR_Exit:

```

; restore
SWAPF STATUS_TEMP,W ;un-swap STATUS into W
MOVWF STATUS ;restores flags AND bank bits (RPO/RPL) } restore
SWAPF W_TEMP,W ;swap W_TEMP nibbles in place
SWAPF W_TEMP,W ;swap back into W, STATUS untouched
RETFF

```

End

Figure 5: ISRHandler save and restore

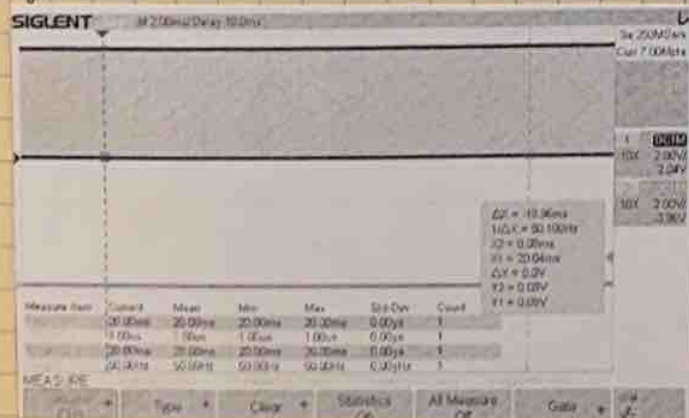
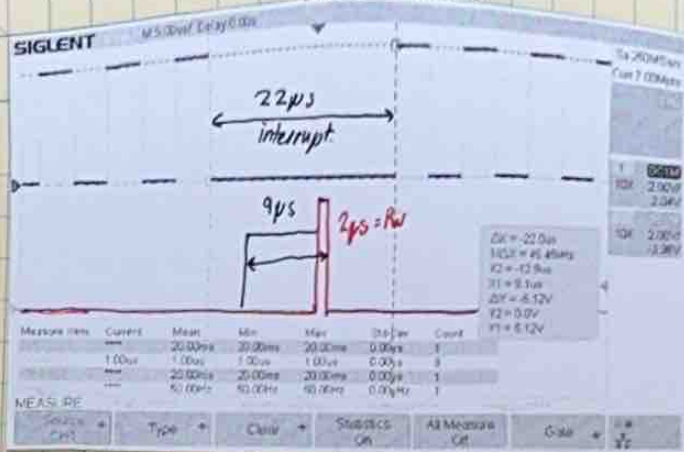


Figure 6: 20 ms period capture that was captured on the oscilloscope

- The frequency counter measured value = 19.999346ms
- oscilloscope = 19.96ms
- predicted = 20ms.



RA0

• When interrupt happens and the next count into the 20ms I am only 9/16 into the count.

RA1

• Figure 7: RA0 main toggle interrupt capture with RA1 toggle to prove interrupt



TJR
9-28-26

• Figure 8: Frequency counter capture of RA1 period

- The count because of the 1/16th prescaler from the ISR vector to the toggle bit is 4 counts into the full 16 count so the count is 9/16's of a full count.

• Part 2 - Intersection State Machine Logic

• Goal:

- Build and demonstrate the traffic-light state machine without using a hardware timer. Use the dedicated external interrupt INT as a manual state-advance event. Each valid external interrupt increments a 4 bit STATE_COUNT field in intersection_state. Use PORTB interrupt-on-change to latch car-detection events. Main evaluate the packed state and decides whether the current traffic direction remains selected or begins a transition. On every iteration of main, display the contents of intersection_state on PORTC so the state byte can be observed on LED's. The purpose of this part is to make the state machine logic observable as events occur and the state changes. Timing and actual traffic-light outputs are added later.

• Required state register:

- Use one GPR named intersection_state. Choose and document its GPR address. Use the register map ↓

Bit	7	6	5	4	3	2	1	0	LG
Name	SC3	SC2	SC1	SC0	TRANSITION	EW_DETECTED	NS_DETECTED	DIRECTION	

• Table 1: Used register map for intersection_state

LG

The ^{LSB} upper nibble is a 4-bit state-count field. The lower nibble contains the intersection flags. ^{LSB}

Field	Meaning
STATE_COUNT[3:0]	current manual state-machine count
DIRECTION	0 = N/S direction; 1 = E/W direction
NS_DETECTED	a low-to-high N/S car-detection event has been latched
EW_DETECTED	a low-to-high E/W car-detection event has been latched
TRANSITION	0 = selected direction is green; 1 = selected direction is yellow

Table 2: intersection_state register map direction / detection / transition for Port 2 - See pg 94 for more...

Initial state:

- Begin with

- STATE_COUNT = 0
 - DIRECTION = 0 for N/S
 - ~~TRANS~~ ^{LSB} TRANSITION = 0
 - both car-detection flags clear
- } This is how it starts up.

- Main must copy this initial intersection_state value to PORTC

External state-advance interrupt:

- Use the dedicated external interrupt as the state-advance input. Each valid external interrupt must increment only STATE_COUNT in bits 7:4 of intersection_state. STATE_COUNT must be incremented as part of a read-modify-write operation on the packed state byte. The lower four state flags must remain unchanged by the increment.

- Document the external-interrupt configuration and the mask/operations used to:

- extract STATE_COUNT
- position it for incrementing
- increment it
- place the updated value back into bits 7:4
- preserve bits 3:0 in intersection_state.

PORTB car detection:

- Use PORTB interrupt-on-change for two car-detection inputs:

- N/S car detection: a low-to-high change sets NS_DETECTED.
- E/W car detection: a low-to-high change sets EW_DETECTED.

- Once set, a car detection flag remains set until the state machine clears it. Repeated detection in the same direction don't count additional cars. The corresponding flag simply remains set.

PORTC state display: on every int iteration of main -

- 1) copy the complete intersection_state byte to PORTC
 - 2) then evaluate STATE_COUNT and the State flags
 - 3) update intersection_state only when the current state requires an action.
- PORTC is therefore a direct live display of the packed state bytes

State-count decisions:

Main must examine STATE_COUNT and decide what action or test is required.

The required decision points are:

STATE_COUNT	Required behavior
0	do nothing
1	do nothing
2	do nothing
3	evaluate <u>DIRECTION</u> , <u>NS_DETECTED</u> , <u>EW_DETECTED</u> , and <u>TRANSITION</u> to decide whether to remain in the current direction or begin a transition
4	finish an active transition

Table 3: STATE_COUNT required action of specific points

Direction decision at STATE_COUNT=3:

- At STATE_COUNT=3, use all four stage flags to decide whether the selected traffic direction changes.

Develop the decision logic based on the following rules:

N/S detected	E/W detected	Action
1	0	remain N/S
0	1	begin transition
1	1	begin transition
0	0	begin transition

Table 4: Table/truth table for N/S green

N/S detected	E/W detected	Action
0	1	remain E/W
1	0	begin transition
1	1	begin transition
0	0	begin transition

Table 5: Truth table for E/W green

- If the decision is not changed "direction", reset to a fresh state in the same direction:

- Clear STATE_COUNT
- Clear NS_DETECTED
- Clear EW_DETECTED

- If the decision is change-direction:

- Set TRANSITION

- If TRANSITION is already set, do nothing.

- Develop and simplify the logic, then document your complete truth table and simplification process.

T	D	NS	EW	Action
0	0	0	0	change direction
0	0	0	1	change direction
0	0	1	0	same direction
0	0	1	1	change direction
0	1	0	0	change direction
0	1	0	1	same direction
0	1	1	0	change direction
0	1	1	1	change direction
1	X	X	X	do nothing (already transitioning)

- Verified by using the K map method and were same direction has the states at change direction and it comes down to button press selected.

$$\text{Same} = \overline{NS} + \overline{EW}D \quad \text{NS} \overline{EW} \overline{D} + \overline{EW} \overline{NS} D$$

$$\text{Change} = \overline{T} + \text{everything else}$$

$$\text{do nothing} = T$$

- each state is all different and unique.

Table 6: Simplified truth table for direction change

• Transition completion at STATE_COUNT=4:

- When STATE_COUNT=4, complete the transition to the new direction and reset to a fresh state:

- toggle DIRECTION
- clear TRANSITION
- clear NS_DETECTED
- clear EW_DETECTED
- clear STATE_COUNT

• Shared-state warning:

- The external-interrupt service, PORTB IOC service, and main loop all use intersection_state. Because several parts of the program can modify different fields within the same byte, the packed state byte can be corrupted if updates interfere with one another. Your implementation must prevent state corruption. Determine and document my solution.

• Before Lab:

- external-interrupt input circuit and configuration
- two PORTB car-detection inputs and loading/electrical analysis.
- external interrupt and PORTB IOC SFR documentation
- intersection_state register map
- masks and packed-field operations for STATE_COUNT and the flags.
- complete state-machine flowchart that accounts for every STATE_COUNT value your program can encounter.

- complete four flag truth table and document logic simplification for the state_count=3 decision.
- source structure showing the read-modify-write operations on the packed state byte
- source code.

LG

In the Lab:

- 1) Verify the initial packed state with $STATE_COUNT=0$ and confirm the main copies intersection_state to PORTC.
- 2) Trigger the external interrupt and verify that only $STATE_COUNT$ changes.
- 3) Advance through $STATE_COUNT=0, 1, \& 2$ and confirm that main takes no state action other than displaying the current packed state on PORTC.
- 4) Verify a low-to-high N/S car detection event sets $NS_DETECTED$.
- 5) Verify a low-to-high E/W car detection event sets $EW_DETECTED$.
- 6) Verify high-to-low sensor changes do not set the detection flags.
- 7) Implement and test the $STATE_COUNT=3$ logic. Test and document all 16 combinations of the four state flags.
- 8) Implement and test the $STATE_COUNT=4$ transition completion logic.
- 9) Repeat several complete manual state-machine cycles while observing the packed state on PORTC.
- 10) Stress the design with external-interrupt and IOC events occurring close together and verify the packed state remains valid.

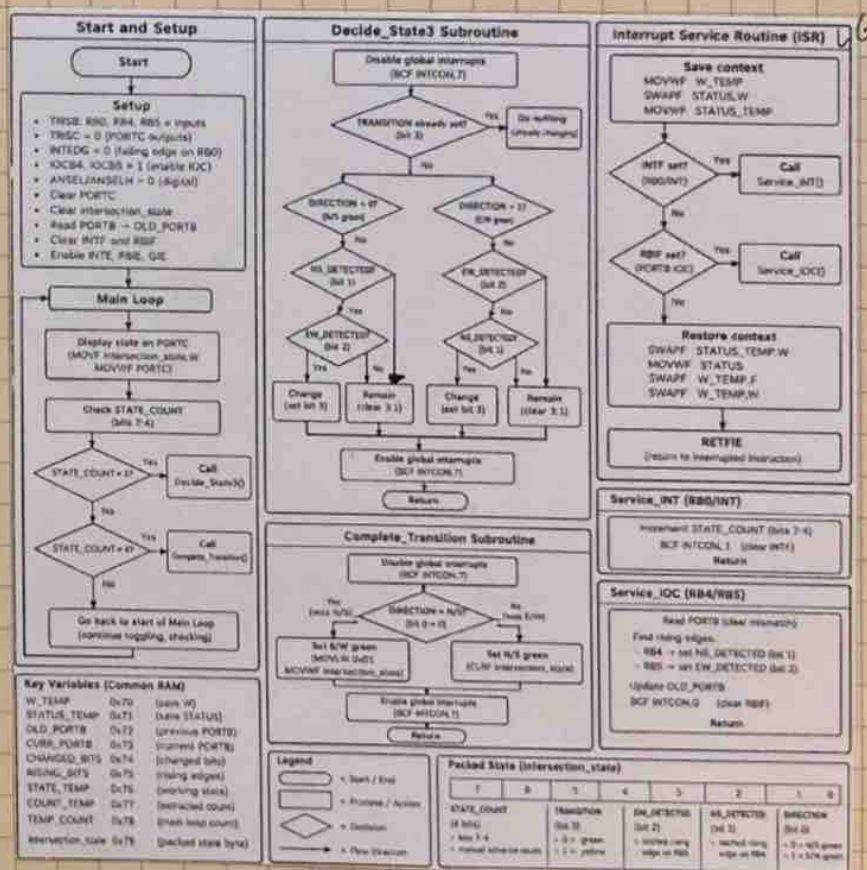


Figure 9: Intersection State Machine Logic flow chart

-PORTB set as inputs, PORTC as outputs, setup interrupts, and the initial intersection state. The main loop displays the current state on PORTC and checks the STATE_COUNT. When the interrupt routine handles either the RBD state-advance button or the RBA/RBS car detected sensors/buttons, updates the appropriate state information, and then returns to the main program.

```

Setup:
; Bank 1
BANKSEL TRISB
BSF TRISB,0 ;set RB0 as input
BSF TRISB,4 ;set RB4 as I/O sensor input
BSF TRISB,5 ;set RB5 as I/O sensor input
CLRF TRISC ;portC as I/O outputs for LEDs
BANKSEL OPTION_REG
BCF OPTION_REG,4 ;interrupt on a falling edge on RB0/INT
BSF IOCB,4 ;watch RB4 for changes
BSF IOCB,5 ;watch RB5 for changes
; Bank 2
BANKSEL ANSEL
CLRF ANSEL
CLRF ANSELH ;set digital I/O
; Bank 0
BANKSEL PORTC
CLRF PORTC ;clear portC outputs to a known state
CLRF intersection_state ;set STATE_COUNT=0, DIRECTION=0 (N/S), TRANSITION=0, flags cleared
MOVWF PORTB,W ;set a baseline PortB reading
MOVWF OLD_PORTB ;reports the sensor that actually changed

BCF INTCON,0 ;clear the change on PortB interrupt flag
BCF INTCON,1 ;clear the external interrupt flag
BSF INTCON,3 ;enable change on PortB interrupt enable bit
BSF INTCON,4 ;enable external interrupt enable bit
BSF INTCON,7 ;global interrupts enable

```

configure I/O

interrupt on change for PortB

setup I/O

clear PortC to start at 0

enable interrupts & clear flags.

Figure 10: SFR's for Intersection State Machine

```

Main:
MOVWF intersection_state,W
MOVWF PORTC ;display the complete packed state, every pass

SWAPF intersection_state,W
ANDLW 0x0F ;isolate STATE_COUNT into the low 4 bits
MOVWF TEMP_COUNT

MOVWF TEMP_COUNT,W
XORLW 3
BTFSB STATUS,2 ;test if STATE_COUNT 3
CALL Decide_State3

MOVWF TEMP_COUNT,W
XORLW 4
BTFSB STATUS,2 ;test if STATE_COUNT 4
CALL Complete_Transition
GOTO Main

```

Figure 11: Main code for checking changes in the intersection

```

ISRHandler:
; save
MOVWF W_TEMP ;save W (no flags affected)
SWAPF STATUS,W ;STATUS into W with nibbles swapped, no flags change
MOVWF STATUS_TEMP ;save swapped STATUS

; Button caused interrupt
BTFSB INTCON,1 ;if RB0 is selected
CALL Service_INT

; sensor change caused this
BTFSB INTCON,0 ;if RB4 or RB5 is selected
CALL Service_IOC

; restore
SWAPF STATUS_TEMP,W ;un-swap STATUS into W
MOVWF STATUS ;restore flags and bank bits (RPO/RPI)
SWAPF W_TEMP,W ;swap W_TEMP nibbles in place
SWAPF W_TEMP,W ;swap W_TEMP into W, STATUS untouched
RETFIE

```

Figure 12: ISR Handler to Identify which interrupt occurred

These sections are like the old code and then new code used to make the State machine logic. The TEMP_COUNT is a temporary holding spot in the RAM in main uses each pass to read out STATE_COUNT and make it a normal number and uses it. Once used it gets discarded and holds no data after it is used.

```

Decide_State3:
    BCF    INTCON, 7           ;protect this multi-step read/write -disable interrupts L6
    BTFSC  Intersection_state, 3 ;check if TRANSITION is already set
    GOTO   D3_Done             ;yes already changing, do nothing

    BTFSS  Intersection_state, 0 ;test DIRECTION
    GOTO   D3_EW_Green         ;DIRECTION=1 -> E/W green path
                                ;DIRECTION=0 -> N/S green:

    BTFSS  Intersection_state, 1 ;NS DETECTED?
    GOTO   D3_Change           ;NS not detected -> change (0,0 / 0,1 cases)
    BTFSC  Intersection_state, 2 ;EW DETECTED also set?
    GOTO   D3_Change           ;both detected -> change (1,1 case)
    GOTO   D3_Remain           ;NS only -> remain (1,0 case)

D3_EW_Green:
    BTFSS  Intersection_state, 2 ;was EW DETECTED
    GOTO   D3_Change           ;EW not detected -> change } check if E/W changed
    BTFSC  Intersection_state, 1 ;was NS DETECTED also set?
    GOTO   D3_Change           ;both detected -> change } check if N/S changed
    GOTO   D3_Remain           ;EW only -> remain

D3_Remain:
    MOVF   Intersection_state, W ;keep DIRECTION bit0 and TRANSITION(bit3, =0 here)
    ANDLW  0x09                 ;clears STATE_COUNT, NS_DETECTED, EW_DETECTED } if no change
    MOVWF  Intersection_state
    GOTO   D3_Done

D3_Change:
    BSF    Intersection_state, 3 ;set TRANSITION about to change to yellow - if changed.

D3_Done:
    BSF    INTCON, 7           ;turn interrupts back on - re-enable interrupts
    RETURN

Complete_Transition:
    BCF    INTCON, 7           ;protect the rewrite
    BTFSS  Intersection_state, 0 ;which direction was green?
    GOTO   CT_ToEW             ;was N/S - switch to E/W
    CLRF   Intersection_state   ;was E/W - new state is all zero (N/S green)
    GOTO   CT_Done

CT_ToEW:
    MOVWF  0x01
    MOVWF  Intersection_state   ;new state: E/W green, everything else 0
    GOTO   CT_Done

CT_Done:
    BSF    INTCON, 7           ;interrupts back on - re-enables interrupts
    RETURN

```

Figure 13: Decide State 3 to check direction to change color or stay.

- This section looks at current direction selected and checks for the "sensors" was detected. Then decides whether to stay green or start a transition or change.

```

Service_INT: MOVWF 0x10
; STATE_COUNT into low nibble
; add 0x10 which effects 7-4 and increments STATE_COUNT 0 through 15
; clear button flag
RETURN

Service_IOC:
    MOVF   PORTB, W           ; read PORTB (always the same)
    MOVWF  CURR_PORTB, W
    XORWF  OLD_PORTB, W       ; W = bits that changed
    ANDWF  CURR_PORTB, W     ; W = changed AND low-high = rising edges only
    MOVWF  RISING_BITS

    BTFSC  RISING_BITS, 4     ; RB4 (N/S) rose?
    BSF    Intersection_state, 1 ; latch NS_DETECTED

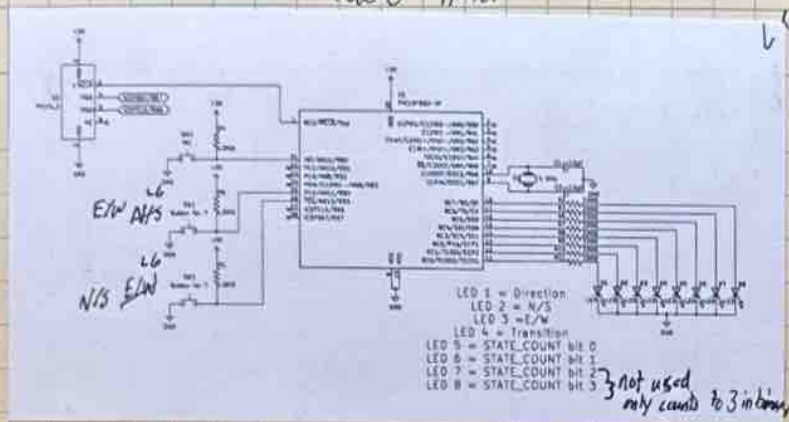
    BTFSC  RISING_BITS, 5     ; RB5 (E/W) rose?
    BSF    Intersection_state, 2 ; latch EW_DETECTED

    MOVF   CURR_PORTB, W
    MOVWF  OLD_PORTB
    BCF    INTCON, 0         ; clear second flag
    RETURN

```

Figure 14: Services the ISR handler to check what interrupt occurred and do it.

- The Service sections checked in the ISR handler and come here. INT just increments the last bits in port C. IOC actually reads PORTB current change, checks if it was RB4 (N/S) or RB5 (E/W) changed.



• Figure 15: Full schematic for Part 2

• Intersection State -

- 7-4 STATE_COUNT = incremented by button interrupt BBO/INT used to decide when to evaluate sensors (count=3) or finish a transition (count=4)

- 3 Transition = 1 = currently changing
- 2 E/W DETECTION = 1 = East/West button has seen a new change (IOC) PORTB.
- 1 N/S DETECTION = 1 = North/South button has seen a new change (IOC) PORTB.
- 0 Direction = 0 = North/South has green, 4 = East/West has green

- External interrupt changes only STATE_COUNT -

BBO/INT its only job is to increment, it has no control of the cars or direction the light will go. That is determined in Decide STATE 3, not the ISB. The count increments in the Service INT portion of code. STATE_COUNT=3 means it will check all flags to decide whether to keep current green or start changing.

- low to high car detection event is latched -

- PORTC only lets us know a change happened on PORTB not which pin⁶⁶ direction. The Service I/OB figures out which direction was selected. it XORs the current PORTB reading against the previous to find the change, the ANDs it to keep only the high bits. the high to low ~~change~~ worry is removed because the AND. These stay until it is cleared or changed.

- STATE_COUNT=4 held in Complete Transition -

- When STATE_COUNT=4 is the moment the yellow phase is over finish switching directions. Which toggles the direction, clears transition back to 0, clears flags, and resets STATE_COUNT to 0. Now a completely fresh new start force new direction. When it equals 3 is the only time it can decide what to do. 4 executes the decision made in 3.

- Verified Working operation with Tim.

- When E/W is set high and toggled the Direction light is set high. When N/S is set high and toggled the direction light is set low.

- The current and resistance was chosen to protect and drive. The LEDs need enough current to even turn on. The current used is 15.151mA which is still ok for the PIC. $(5/330\Omega) = 15.152mA$

TJR
4-30-26

Part 3 - State Machine Intersection Timing

Goal:

- Use the packed state byte stored in the intersection_state register to track and control the intersection timing. The STATE_COUNT will now be incremented by a timer ISR. Choose a single timer interval and STATE_COUNT range that allows both 5-second and 1-second intervals to be represented. PORTA will display the actual traffic light signals with the correct color LED's. PORTC will display the realtime state byte. Car detection will be ignored in this section. This section focuses on state driven timing. Much of this code will be reused in later parts. Consider how to structure source so the state machine logic is clear, encapsulated and reusable. The selected direction remains green for 5 seconds. The transition lasts 1 second.

Required timing:

Normal operation alternates:

- N/S green, E/W red → 5 seconds
- N/S yellow, E/W red → 1 second
- N/S red, E/W green → 5 seconds
- N/S red, E/W Yellow → 1 second
- repeat

Timer-driven STATE_COUNT:

- Reuse the packed intersection_state register and state-field definitions from Part 2.

In Part 3:

- the dedicated external interrupt is no longer used to advance STATE_COUNT
- PORTB car-detected logic is not used
- NS_DETECTED & EW_DETECTED remain clear
- one selected hardware timer generates the periodic interrupt that advances STATE_COUNT

- Each timer interrupt increments only STATE_COUNT in bits 7:4 of intersection_state. The lower four state flags must be preserved during the packed read-modify-write operation. The timer ISR should service the timer, increment STATE_COUNT, and return. It doesn't decide which traffic lights should be on.

Select one timing interval:

- Choose one hardware timer and one periodic interrupt interval for all normal intersection timing in this part.

Using that one timer interval, determine the STATE_COUNT value that represents:

- 5 seconds of green
- 1 second of yellow

- Both durations must be represented by whole-number multiples of timer interrupts and both required count values must fit within the 4-bit STATE_COUNT range of 0 through F. You do not need to use the entire 0-F range.

- Timer Selected: Timer 1 due to its long period 16-bit
- the Timer configuration: 1:8 prescaler, postscaler 16
- the timer interrupt-period calculation prebad $T_{MR1L} = 0 \times 0B0C$ more sec pg
- the selected periodic interval
- the number of timer interrupts required for 5 sec → 10 interrupts @ 0.5 sec interrupts
- the number of timer interrupts required for 1 sec → 2 interrupts @ 0.5 sec interrupts

• State-COUNT map:

- Map out every STATE_COUNT value from 0 through the highest count used by your design before implementing the timed state machine. For each count value, document the elapsed time and required behavior.

STATE_COUNT	Elapsed time	Behavior
0-9	0.0s - 4.5s	nothing (green holding)
10	5.0s	evaluate — with car detection off, this always sets TRANSITION
11	5.5s	nothing (yellow holding)
12	6.0s	complete transition — toggle DIRECTION, reset everything to 0

Table 7: STATE_COUNT for timer usage in Part 3 State Machine

• Timed state machine rules:

- Reuse the state-transition actions developed in Part 2, but apply them at the timer derived count boundaries from Part 3 STATECOUNT map rather than at the literal Part 2 counts. In this section, the car detection flags remain clear and are ignored.

• PORTA traffic light counts⁶ outputs:

- Use Port A to drive six LED's representing:

- N/S red
- N/S yellow
- N/S green
- E/W red
- E/W yellow
- E/W green

- Choose and document the PORTA bit assignment for each LED. Main must decode DIRECTION and TRANSITION into the correct traffic-light outputs

- The four legal normal output states are:

DIRECTION	TRANSITION	N/S lights	E/W lights
0	0	Green	Red
0	1	Yellow	Red
1	0	Red	Green
1	1	Red	Yellow

Table 8: The states for Direction and transition with what light is with it

- Document the exact PORTA binary/hex patterns for each state. The current traffic light pattern must be updated on every iteration of main based on the current DIRECTION and TRANSITION values.

• PORTC state display:

- PORTC must continue to display the completed⁶⁶ packet intersection state byte on every iteration of main.

• Shared-state behavior:

- The timer ISR and main both access intersection state. Reuse or adapt the state-protection method developed in Part 2 so timer-driven STATE_COUNT updates and main-loop flag changes do not corrupt the packet state byte. ⁶⁶Document changes required when the module external-interrupt state advance from Part 2 is replaced by periodic timer interrupts.

• Before Lab:

• selected hardware timer and reason for the choice

- complete timer interrupt-period calculation and configuration
- calculated STATE_COUNT values for the 5-second & 1 second intervals
- complete STATE_COUNT map for every value from 0 through the highest count used
- timer and interrupt SFR documentation
- intersection state register map from Part 2
- packet-field masks and read-modify-write operations used to increment and clear STATE_COUNT
- shared-state protection method reused or adapted from Part 2
- PORTA traffic-light schematic and loading/electrical analysis
- PORTA bit assignments and the binary/hex value for all four legal traffic-light states
- updated ISR flowchart
- complete timed state machine flowchart
- source code

• In the Lab:

- 1) Disable the Part 2 external-interrupt state advance and car-detection behavior
- 2) Verify both car detection flags remain clear
- 3) Verify the selected timer generates the periodic interrupt interval calculated
- 4) Observe STATE_COUNT increment on PORTC and verify the lower four state bits remain unchanged by the timer ISR.
- 5) Observe or otherwise verify the complete STATE_COUNT range used by design and confirm that each value behaves as documented in state map.
- 6) Verify N/S begins green with E/W red
- 7) Measure the N/S green interval and verify it lasts 5 seconds
- 8) Verify the state COUNT changes to N/S yellow, transition safe, and STATECOUNT resets for yellow interval.
- 9) Measure the N/S yellow interval and verify it lasts 1 second.
- 10) Verify the yellow interval completes by toggling Direction, clearing TRANSITION, clearing STATE_COUNT, and beginning E/W green.
- 11) Repeat the same checks for the E/W green and yellow states.
- 12) Observe PORTA traffic-light outputs with packet state shown on PORTC.
- 13) Verify no state produces conflicting green outputs.
- 14) Compare measured 5 sec & 1 sec intervals with calculated and % error.
- 15) Verify the packet state remains valid when a timer interrupt occurs near main loop state updates.

; config statements should precede project file includes.
#include <xc.inc>
W_TEMP EQU 0x70 ;save W during ISR
STATUS_TEMP EQU 0x71 ;save STATUS during ISR
TEMP_COUNT EQU 0x72 ;STATE_COUNT read out in main
intersection_state EQU 0x73 ;the packed state byte

; Reset Vector at 0000h. Execution starts here after reset.
PSECT resetVect, class=CODE, delta=2
ResetVector:
GOTO Setup

; Interrupt Vector at 0004h. Execution starts here after an interrupt.
PSECT isrVect, class=CODE, delta=2
InterruptVector:
GOTO IsrHandler

Figure 18: Include section for registers and the interrupt vector

Setup:
; Bank 1
BANKSEL TRISA
CLRF TRISA ;RA0-RA5 all outputs (six traffic-light LEDs)
CLRF TRISC ;PORTC all outputs (state display)
BSF PIR1,0 ;enable Timer1 interrupt (TMR1IE)
; Bank 3
BANKSEL ANSEL
CLRF ANSEL ;RA0-RA5 digital
CLRF ANSELH
; Bank 0
BANKSEL PORTA
CLRF PORTA ;lights off - first main pass sets the real pattern
CLRF PORTC
CLRF intersection_state ;count=0, N/S green, no transition, flags clear
MOVLW 0x08
MOVWF TMR1H ;preload for a 500 ms period, high byte
MOVLW 0xDC
MOVWF TMR1L ;preload low byte
BCF PIR1,0 ;clear TMR1IF before starting
BCF INTCON,4 ;Part 2's external INT stays off in Part 3
BCF INTCON,3 ;Part 2's PORTB IOC stays off in Part 3
MOVLW 0b00110001 ;setup prescale 1:8, internal clock, Timer1 ON
MOVWF T1CON ;turn on the prescaler and Timer1
BSF INTCON,6 ;Timer1 is a peripheral interrupt
BSF INTCON,7 ;global interrupts enable

Main:
MOVE intersection_state, W
MOVWF PORTC ;what state of the LEDs on PortC, every pass
CALL Decode_Lights ;keeps or changes based on what occurs every pass
SWAPF intersection_state, W
ANDLW 0x0F ;pull STATE_COUNT into TEMP_COUNT ANDLW 0x0F
;then wipes out the garbage left in the top half
MOVWF TEMP_COUNT ;save the ANDLW in TEMP_COUNT to compare
MOVF TEMP_COUNT, W
XORLW 10
BTFSK STATUS, 2 ;count is 10 which will be 5.0 s of green elapsed
CALL Decide ;after 5s we need to decide to change to yellow
MOVF TEMP_COUNT, W
XORLW 12
BTFSK STATUS, 2 ;count is 12 which will be 1.0 s of yellow elapsed
CALL Finish ;after 1s we need to decide to what direction to change
GOTO Main

Figure 19: Main code and Setup for operation of State Machine SFR

```

Decide:
BCF    INTCON,7           ;Turn of the global interrupt to make sure operation happens
BTFS   intersection_state,2 ;is TRANSITION bit set high = skip
GOTO   Done              ;dont do anything
BTFS   intersection_state,0 ;look which direction is green N/S=0, E/W=1
GOTO   NS_Green

EW_Green:
BTFS   intersection_state,2 ;is E/W DETECTED bit set high = skip
GOTO   Change            ;if low goto change
BTFS   intersection_state,1 ;is N/W DETECTED bit set high = skip
GOTO   Remain            ;goto Remain if N/W bit is low
GOTO   Change            ;goto Change if N/W and E/W bits high

NS_Green:
BTFS   intersection_state,1 ;is N/S bit set high = skip
GOTO   Change            ;if N/S bit low then goto change
BTFS   intersection_state,2 ;is E/W bit set high = skip goto change
GOTO   Remain            ;if E/W low there is no car, N/S has traffic

Change:
BSF    intersection_state,2 ;turn On the TRANSITION bit to switch output pattern yellow
GOTO   Done

Remain:
BTFS   intersection_state,0 ;check DIRECTION again before we clear the register
GOTO   NS_Remain          ;we keep N/S green

EW_Remain:
CLRF   intersection_state ;clear everything
BSF    intersection_state,0 ;only select direction E/W
GOTO   Done

NS_Remain:
CLRF   intersection_state ;clear the register for a fresh start

Done:
BSF    INTCON,7           ;turn on the interrupts
RETURN                    ;go back to main where Decide was called from

;Finish: runs when STATE_COUNT hits 12 - the 1 second of yellow
;is over. Swap which direction is green and reset everything.

Finish:
BCF    INTCON,7           ;protect this the same way Decide does
BTFS   intersection_state,0 ;which way was green before this?
GOTO   ToEW              ;bit was 0 - was N/S, switch to E/W
CLRF   intersection_state ;bit was 1 - was E/W, back to N/S green, count reset to 0
GOTO   FinishDone

ToEW:
MOVLW  0x01
MOVWF  intersection_state ;now E/W green, everything else reset to 0

FinishDone:
BSF    INTCON,7           ;interrupts back on
RETURN

```

*Figure 20: Check what direction is set high to know what change happens next.

-in this section it clarifies which selection of lights is set. First checks what green light is set and then checks the yellow state. So then it can switch the lights direction.

-Due to the required timing which starts with:

• N/S green, E/W red = 5sec

• N/S yellow, E/W red = 1sec

• N/S red, E/W green = 5sec

• N/S red, E/W yellow = 1sec

-Then repeat over and over again

Decode_Lights: looks at *DIRECTION* and *TRANSITION* and turns on the matching traffic lights on *PORTA*. Called every pass through *Main* so *PORTA* always matches the current state.

Decode_Lights:

```
BTFSF intersection_state,3 ;is TRANSITION yellow flag on
GOTO Green_Phase ;no we're in a green/red phase
GOTO Yellow_Phase ;yes we're in a yellow/red phase
```

Green_Phase:

```
BTFSF intersection_state,0 ;which way is green?
GOTO Set_NSGreen ;bit was 0 N/S green
GOTO Set_EWGreen ;bit was 1 E/W green
```

Yellow_Phase:

```
BTFSF intersection_state,0
GOTO Set_NSYellow
GOTO Set_EWYellow
```

Set_NSGreen:

```
MOVLW 0b00001100 ;RA1 on (N/S green), RA3 on (E/W red)
GOTO Store_Lights
```

Set_EWGreen:

```
MOVLW 0b00100001 ;RA0 on (N/S red), RA5 on (E/W green)
GOTO Store_Lights
```

Set_NSYellow:

```
MOVLW 0b00001010 ;RA1 on (N/S yellow), RA3 on (E/W red)
GOTO Store_Lights
```

Set_EWYellow:

```
MOVLW 0b00010001 ;RA0 on (N/S red), RA4 on (E/W yellow)
```

Store_Lights:

```
MOVWF PORTA ;takes moved values and then displays on the LEDs
RETURN
```

ISR - runs any time an interrupt fires. Right now the only thing that should be able to interrupt is *Timer1*.

IsrHandler:

```
; save
MOVWF W_TEMP ;save W (no flags affected)
SWAPF STATUS,W ;STATUS into W with nibbles swapped, no flags change
MOVWF STATUS_TEMP ;save swapped STATUS
```

button caused interrupt

```
BTFSF PIR1,0 ;if Timer1 roll over flag set call handler
CALL Service_Timer1
```

restore

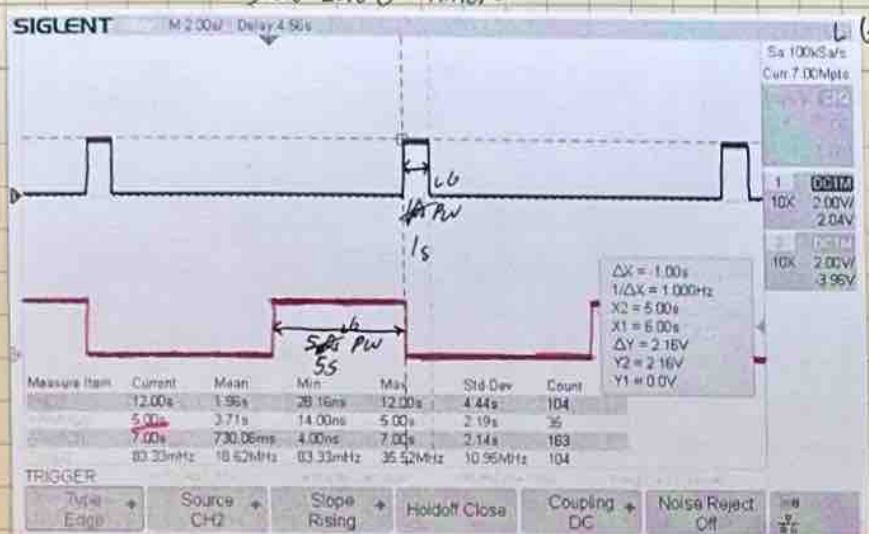
```
SWAPF STATUS_TEMP,W ;un-swap STATUS into W
MOVWF STATUS ;restores flags AND bank bits (RS0/RS1)
SWAPF W_TEMP,W ;swap W_TEMP nibbles in place
SWAPF W_TEMP,W ;swap back into W, STATUS untouched
RETFIE
```

Service_Timer1:

```
BCF PIR1,0 ;clear Timer1 flag
MOVLW 0x0B ;reload timer 1 with the 500ms count
MOVWF TMR1H ;load into the higher Timer1 bits
MOVLW 0xDC ;reload timer one with the rest of the count
MOVWF TMR1L ;load into the lower Timer1 bits
MOVWF 0x10 ;increment the upper bits if intersection_state
ADDWF intersection_state,F ;add increments the upper bits
RETURN ;return to ISR
```

Figure 21: Which select lights are on and *ISR* handler with the timer service

The reason for reloading the register of *TMR1H* & *TMR1L* is because a new count has occurred and it needs to start at 3036 to be 500,000 μ s or 500ms and that way the count works as desired.



TJR
10-1-26

Figure 22: Capture of 5s and 1s green N/s and yellow N/s

Part 4 - Complete intersection

Goal:

- Combine the state machine and car-detection logic from Part 2 with the timer driven sequencing and traffic-light outputs from Part 3 to create the complete working intersection. Don't redesign those sections from scratch. Reuse the working logic, packed intersection state, timer configuration, car-detection behavior, PORTA traffic-light outputs, PORTC state display, and shared-state protection already developed and documented.

- Add one new feature: use an other wise unused PORTB input as a RUN/DEBUG mode selector

• Run mode: the selected hardware timer advances STATE_COUNT.

• DEBUG mode: the dedicated external interrupt INT advances STATE_COUNT manually.

• The state machine itself must behave the same in either mode. Only the source of the state count increment changes.

RUN/DEBUG mode selection:

- Choose and document one unused PORTB pin for the mode selector. (PB3) The mode selector input is not a car sensor and must not alter either car detection flag. Choose and document which logic level represents RUN and which represents DEBUG.

- The required behavior is:

Mode	Timer may advance STATE_COUNT	INT may advance STATE_COUNT
RUN	Yes	No
DEBUG	No	Yes

Table 9: RUN/DEBUG mode selection table

- Changing the mode-selector input must not increment STATE_COUNT by itself. When changing between modes:

• retain the current intersection state

• retain the current traffic light state

• retain any latched car-detection flags

• ensure only the selected state advance source can change STATE_COUNT

• prevent stale or pending interrupt conditions from causing an unintended increment.

• Complete intersection behavior:

- The complete Part 4 program must combine the behavior already established in the previous parts:

- use the Part 2 packed state byte and car-detection decision logic.
- use PORTB IOC to latch the N/S and E/W car-detection events as developed in part 2.
- use the Part 3 timer configuration and count values for the 5-second green and 1 sec yellow intervals
- use the Part 3 PORTA traffic light output decoding
- continue displaying the complete intersection-state byte on PORTC on every main-loop iteration
- preserve the shared packed state correctly when interrupts and main modify different fields.

- In RUN mode, the intersection operates continuously using the timer. In DEBUG mode, the same state machine is stepped manually with INT, allowing individual state-count changes, car detections, and state decisions to be observed and tested.

- The Part 2 car-detection rules apply at the end of the green interval, using the Part 3 STATE_COUNT value that represents 5 sec rather than the literal Part 2 count of 3. If the decision is to transition, use the Part 3 1sec transition timing before changing direction. Refer to Parts 2 and 3 rather than creating a second implementation of the same logic.

• Before Lab:

- completed Part 2 state machine logic, truth table, car-detection inputs and packed-state operations
- completed Part 3 timer configuration, timing calculations, STATE_COUNT map, PORTA traffic-light map, and timed state-machine flow.
- RUN/DEBUG selector circuit, PORTB pin assignment, logic definition, and loading/electrical analysis
- any new SFR/pin documentation required for the mode selector
- an updated ISR/main flowchart showing how the selected mode determines whether the timer or INT may advance STATE_COUNT
- the method used to prevent the unselected interrupt source or a mode change from creating an unintended state increment
- final integrated source code

• In the Lab:

- 1) Verify the RUN/DEBUG selector and confirm changing modes does not itself change STATE_COUNT.
- 2) In DEBUG mode, verify timer interrupts do not advance STATE_COUNT
- 3) In DEBUG mode, use INT to step through the state machine while observing intersection-state on PORTC.
- 4) Use DEBUG mode to verify car detections and the Part 2 end of green decision codes with the actual PORTA traffic-light outputs.
- 5) Verify a manual transition completes correctly and the next direction begins in the correct green state.
- 6) Switch to RUN mode without resetting the program and verify INT no longer advances STATE_COUNT
- 7) In RUN mode, verify the timer advances the same state machine automatically.
- 8) Verify the 5 second green and 1 second yellow timing from Part 3 with car detection active.
- 9) Exercise N/S only, E/W-only, both, and no-car detection cases during normal timed operation and verify the Part 2 decision logic is preserved.
- 10) Stress the complete design with timer, INT, and car detection events occurring near one another
- 11) Verify PORTC continues to show the packed state and PORTA continues to show the corresponding traffic/light state.

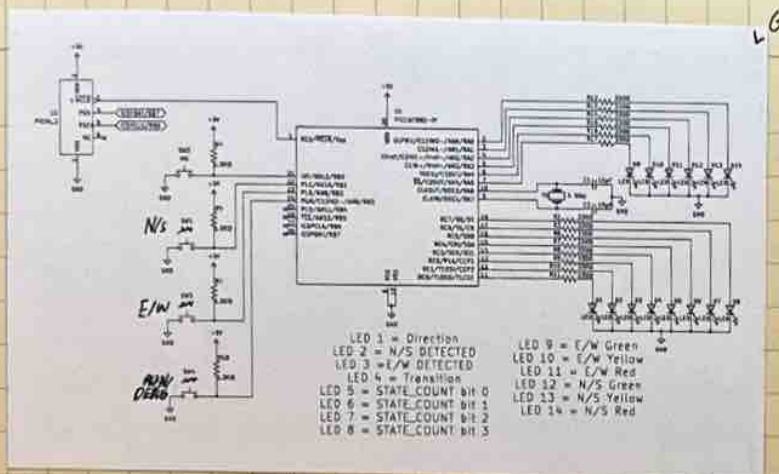


Figure 23: Full schematic for complete intersection.

To start, I combined code of part 2 and Part 3. Part 2 code can be found on pg 92⁹³ and Part 3 on pg 94-101.

Input current = $\frac{5V}{1.5k} = 3.333mA$ well below max

Output current = $\frac{5V}{330} = 15.152mA$ but this is for diodes which need enough to turn on and be bright.

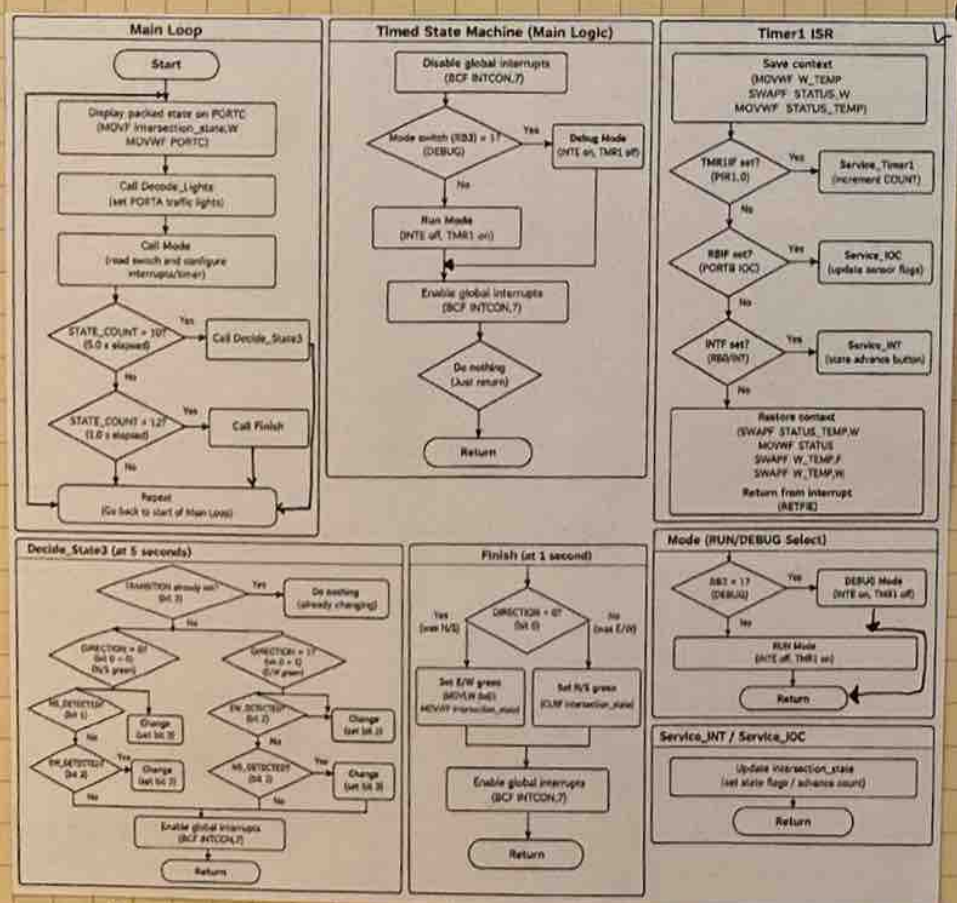


Figure 24: Flow chart for full intersection.

See the flow chart for part 2 & 3 for what all these sections do in detail pg 91 for part 2 and pg 98 for part 3.

Setup:

```

; Bank 1
BANKSEL TRISA
CLRF TRISA ;RA0-RA5 all outputs (six traffic-light LEDs)
BSF TRISB,0 ;set RB0 as input
BSF TRISB,1 ;set RB1 as N/S sensor input
BSF TRISB,2 ;set RB2 as E/W sensor input
BSF TRISB,3 ;set RB3 as the RUN/DEBUG to switch
CLRF TRISC ;portC all outputs for LEDs
BANKSEL OPTION_REG
BCF OPTION_REG,6 ;interrupt on a falling edge on RB0/INT
BSF IOCB,1 ;watch RB1 for changes
BSF IOCB,2 ;watch RB2 for changes
BSF PIR1,0 ;enable Timer1 interrupt (TMR1IE)

; Bank 3
BANKSEL ANSEL
CLRF ANSEL ;RA0-RA5 digital
CLRF ANSELN

; Bank 0
BANKSEL PORTA
CLRF PORTA ;lights off - first main pass sets the real pattern
CLRF PORTC
CLRF intersection_state ;count=0, N/S green, no transition, flags clear
MOVF PORTB,W ;set a baseline PortB reading
MOVF OLD_PORTB ;reports the sensor that actually changed
MOVLW 0x0B
MOVF TMR1H ;preload for a 500 ms period, high byte
MOVLW 0xDC
MOVF TMR1L ;preload low byte
BCF PIR1,0 ;clear TMR1IF before starting
BCF INTCON,0 ;clear the change on PortB interrupt flag
BSF INTCON,3 ;Part 2's PORTB IOC stays off in Part 3
MOVLW 0b00110000 ;setup prescale 1:8, internal clock, Timer1 ON
MOVF TICON ;turn on the prescaler and Timer1
CALL Mode ;set the INT/TMR1 interrupts and turn on Timer1
;to match the switch at this moment
BSF INTCON,6 ;Timer1 is a peripheral interrupt
BSF INTCON,7 ;global interrupts enable

```

Figure 25: Full setup code for Part 4 Complete Intersection

Main:

```

MOVF intersection_state,W
MOVF PORTC ;what state of the LEDs on PortC, every pass
CALL Decode_Lights

CALL Mode ;keeps or changes based on what occurs every pass

SWAPF intersection_state,W
ANDLW 0x0F ;pull STATE_COUNT into TEMP_COUNT ANDLW 0x0F
;then wipes out the garbage left in the top half
MOVF TEMP_COUNT ;save the ANDLW in TEMP_COUNT to compare

MOVF TEMP_COUNT,W
XORLW 10
BTFSC STATUS,2 ;count is 10 which will be 5.0 s of green elapsed
CALL Decide ;after 5s we need to decide to change to yellow

MOVF TEMP_COUNT,W
XORLW 12 ;2 more interrupts after the 10
BTFSC STATUS,2 ;count is 12 which will be 1.0 s of yellow elapsed
CALL Finish ;after 1s we need to decide to what direction to change
GOTO Main

```

Figure 26: Main code only thing from Part 3 is called Mode

-The mode select code is on pg 106 which decides whether it just runs of the timer or the Debby the manual count.

Mode reads the mode switch and configures which source the button or the timer is allowed to move STATE_COUNT. Called once in Setup and again every pass through Main.

Mode:

```
BCF    INTCON, 7
BTFSC  PORTB, 3      ;bit set then DEBUG, matching your original polarity
GOTO   Debug
```

Run:

```
BCF    INTCON, 4      ;INTF off - button can't move the count in RUN
BCF    INTCON, 1      ;clear any stale INTF
BCF    PIR1, 0        ;clear any stale TMR1IF before starting fresh
BSF    T1CON, 0       ;TMR1ON - timer actually counting
GOTO   ModeDone
```

Debug:

```
BCF    T1CON, 0       ;stop Timer1 outright - its flag can't set while stopped
BCF    PIR1, 0        ;clear any stale TMR1IF
BCF    INTCON, 1      ;clear any stale INTF from before the switch
BSF    INTCON, 4      ;INTF on - button can move the count in DEBUG
```

ModeDone:

```
BSF    INTCON, 7
RETURN
```

Figure 27: Mode selection for operation of the intersection

The RUN will Run on a Low which runs the code like in part 3 and Debug is run during a High and it runs the code like in part 2.

Conclusion:

Lab 6 Timers was used to create a traffic light intersection controlled with the PIC16F883 timers 4 and 2 as well as interrupts. Part one was establishing a base line understanding of how to set up a timer interrupt. By using Timer 2 with a hardware reset a prescaler 1:16, PR2=249, postscale 1:5, which gave a 20ms interrupt with a small 22µs duration to create the 20ms interrupt. Part 2 was creating the intersection but by creating a register intersection state the register held the information of direction, transition, the count, and which direction is selected. The interrupt would increment STATE_COUNT which needed to increment 7:4 of the register. This made it so as it incremented it counts to the count for change. Part 3 was replacing the buttons and now using Timer 1 to make a 5 sec green and a 4 second yellow for the transitioning intersection. Part 2's decision logic was reused with the flags for detection cars cleared. Part 3 was all based on math. Part 4 was combining both Part 3 & Part 2 by making a RUN/DEBUG selector pin to decide whether the timer was used for part 3 or DEBUG to run the external button to increment STATE_COUNT. While the logic itself stayed completely unchanged - which confirms the reusability of the code. Here is the github URL:

https://github.com/groeluthe/Lab_6_Timers-.git

Debugging & troubleshooting was very small context errors but the most annoying problem was for getting to add linker code to properties. Also make sure to enable the correct interrupts or it will freeze the program. Also all was pushed and I have a clean working tree.